



Mémento C++ — Référence Complète

Formation C++ · 5 jours · Toutes les règles, tableaux et exemples en un seul document



Sommaire

Section	Thème
1. Syntaxe fondamentale	Types, variables, opérateurs, contrôle
2. Fonctions	Déclaration, surcharge, lambdas
3. Classes et POO	Encapsulation, constructeurs, RAII
4. Héritage et polymorphisme	virtual, override, interfaces
5. Mémoire et smart pointers	RAII, unique_ptr, shared_ptr
6. STL — Conteneurs	Tableau comparatif et cas d'usage
7. STL — Algorithmes	Référence complète
8. Templates	Fonctions, classes, concepts
9. Exceptions	Hiérarchie, garanties, noexcept
10. C++ Moderne	auto, lambdas, optional, ranges
11. Concurrency	thread, mutex, atomic, async
12. Design Patterns	Tableau de référence
13. CMake	Commandes et structure
14. Tests	Catch2 et GoogleTest
15. Sécurité	Vulnérabilités et corrections
16. Performance	Profiling et optimisation
17. Bonnes pratiques	Règles d'or et anti-patterns
18. Outils	Compilateurs, sanitizers, profilers

1. Syntaxe Fondamentale

1.1 Types de données fondamentaux

Type	Taille	Plage	Suffixe littéral	Exemple
bool	1 oct	true / false	—	bool b = true;
char	1 oct	-128 à 127	'A'	char c = 'A';
unsigned char	1 oct	0 à 255	—	uint8_t b = 255;
short	2 oct	-32768 à 32767	—	short s = 100;
int	4 oct	-2.1G à 2.1G	—	int n = 42;
unsigned int	4 oct	0 à 4.3G	42u	unsigned u = 100u;
long long	8 oct	$\pm 9.2 \times 10^{18}$	42LL	long long l = 1e18;
float	4 oct	~7 décimales	3.14f	float f = 3.14f;
double	8 oct	~15 décimales	3.14	double d = 3.14;
size_t	8 oct	0 à max	42zu	size_t s = v.size();

1.2 Modificateurs et qualificateurs

```
const int MAX = 100;           // valeur immuable (runtime)
constexpr int SZ = 32;        // valeur immuable (compile-time)
static int compteur = 0;       // durée de vie statique
volatile int reg = 0;          // accès non-optimisé (matériel)
mutable int cache = -1;        // modifiable dans une méthode const
[[nodiscard]] int calculer();  // valeur de retour obligatoire
[[deprecated("Utiliser v2")]] void ancienne();
[[noreturn]] void mourir();    // ne retourne jamais
```

1.3 Initialisation — 4 syntaxes

```
int a = 42;           // copie (C compatible)
int b(42);             // directe
int c{42};             // uniforme – RECOMMANDÉE (détecte narrowing)
int d = {42};          // copie uniforme

// Piège : Most Vexing Parse
Widget w();            // ❌ déclare une FONCTION
Widget w{};            // ✅ construit un Widget par défaut
Widget w(args);        // ✅ construit avec arguments

// Narrowing detection
int e{3.14};           // ERREUR : perd la précision
int f = 3.14;          // OK mais silencieux (bug potentiel)
```

1.4 Structures de contrôle

```
// if avec initialisation (C++17)
if (auto it = m.find(key); it != m.end()) { use(it->second); }

// switch avec enum class
switch (couleur) {
    case Couleur::Rouge: /* ... */ break;
    case Couleur::Vert:  [[fallthrough]]; // intentionnel
    case Couleur::Bleu:  /* ... */ break;
    default: break;
}

// Boucles
for (int i = 0; i < n; i++) { }           // classique
for (const auto& elem : conteneur) { }    // range-based (recommandée)
while (condition) { }                   // pré-test
do { } while (condition);                 // post-test (garantit 1 tour)

// Contrôle de boucle
break;    // sortir de la boucle
continue; // passer à l'itération suivante
return;   // sortir de la fonction
```

2. Fonctions

2.1 Syntaxes de déclaration

```
// Prototype
int additionner(int a, int b);

// Définition classique
int additionner(int a, int b) { return a + b; }

// Type de retour déduit (C++14)
auto additionner(int a, int b) { return a + b; }

// Type de retour trailing (C++11)
auto additionner(int a, int b) -> int { return a + b; }

// constexpr (évaluable à la compilation)
constexpr int factorielle(int n) {
    return n <= 1 ? 1 : n * factorielle(n-1);
}
```

2.2 Passage de paramètres — règle de choix

Type	Passage par	Quand
Types primitifs	valeur <code>int x</code>	Toujours
Objets (lecture)	<code>const T& x</code>	Par défaut
Objets (modification)	<code>T& x</code>	Si modification nécessaire
Objets (transfert)	<code>T&& x</code>	Pour move semantics
Pointeurs	<code>T* x</code>	Si nullable (peut être nullptr)
<code>unique_ptr</code>	<code>unique_ptr<T> (valeur)</code>	Transfert de propriété

```
// ✅ Bonnes signatures
void afficher(const std::string& s);           // lecture
void mettreEnMajuscules(std::string& s);       // modification
std::string combiner(std::string a, std::string b); // move-friendly
void prendre(std::unique_ptr<Widget> w);       // transfert de propriété

// ❌ Anti-patterns
void afficher(std::string s);                  // copie inutile
void afficher(std::string* s);                 // préférer const ref
```

2.3 Surcharge et valeurs par défaut

```
// Surcharge (même nom, signatures différentes)
int abs(int x);
double abs(double x);
std::complex<double> abs(std::complex<double> x);

// Valeurs par défaut (à droite uniquement)
void log(std::string msg, int niveau = 1, bool horodater = true);

// ⚠️ Ambiguïté : éviter de mélanger surcharge + valeur par défaut
```

3. Classes et POO

3.1 Squelette de classe complète

```
class MaClasse {
public:
    // — Constructeurs —
    MaClasse();                      // par défaut
    explicit MaClasse(int val);      // explicit évite les conversions implicites
    MaClasse(const MaClasse&);       // copie
    MaClasse(MaClasse&&) noexcept;   // move

    // — Affectation —
    MaClasse& operator=(const MaClasse&);
    MaClasse& operator=(MaClasse&&) noexcept;

    // — Destructeur —
    virtual ~MaClasse();             // virtual si classe de base

    // — Méthodes publiques —
    [[nodiscard]] int getValeur() const; // lecture
    void setValeur(int v);             // écriture avec validation
    void traiter();                   // action

protected:
    void helperProtege();             // accessible aux classes dérivées

private:
    int valeur{0};                   // toujours initialisé
    std::string nom;
    static int compteur;              // partagé entre instances
};
```

3.2 Constructeurs — règles essentielles

```
class Cercle {
    double rayon;
    std::string couleur;

public:
    // ✅ Liste d'initialisation – toujours plus efficace que le corps
    explicit Cercle(double r, std::string c = "noir")
        : rayon(r), couleur(std::move(c)) {
        // Validation dans le corps
        if (rayon <= 0)
            throw std::invalid_argument("Rayon doit être > 0");
    }

    // ✅ Délégation de constructeurs (C++11)
    Cercle() : Cercle(1.0) {}
    Cercle(const std::string& c) : Cercle(1.0, c) {}
};
```

3.3 Rule of Zero vs Rule of Five

```
// ✅ Rule of Zero (recommandée) : déléguer à la STL
class ModèleModerne {
    std::unique_ptr<Data> data;    // géré par unique_ptr
    std::vector<Item> items;      // géré par vector
    std::string nom;             // géré par string
    // Pas besoin de déclarer les 5 membres spéciaux
};

// ✅ Rule of Five (si gestion manuelle de ressource)
class TamponBrut {
    char* data;
    size_t taille;

public:
    explicit TamponBrut(size_t n) : data(new char[n]), taille(n) {}
    ~TamponBrut() { delete[] data; }
    TamponBrut(const TamponBrut& o) : data(new char[o.taille]), taille(o.taille) {
        std::copy(o.data, o.data + taille, data);
    }
    TamponBrut(TamponBrut&& o) noexcept : data(o.data), taille(o.taille) {
        o.data = nullptr; o.taille = 0;
    }
    TamponBrut& operator=(const TamponBrut&); // similaire au constructeur copie
    TamponBrut& operator=(TamponBrut&&) noexcept; // similaire au constructeur move
};
```

3.4 Modificateurs d'accès

Modificateur	Classe elle-même	Classes dérivées	Code extérieur
public	✅	✅	✅

Modificateur	Classe elle-même	Classes dérivées	Code extérieur
protected	✓	✓	✗
private	✓	✗	✗

4. Héritage et Polymorphisme

4.1 Syntaxe et types d'héritage

```
class Animal {
public:
    virtual void parler() const { std::cout << "..."; }
    virtual ~Animal() = default; // ← TOUJOURS virtual pour les bases polymorphiques
};

class Chien : public Animal { // public = "est-un"
    void parler() const override { std::cout << "Woof!"; }
};

// class Chien : protected Animal // rare : partiellement "est-un"
// class Chien : private Animal // = "implémenté avec"
```

4.2 Fonctions virtuelles — tableau de référence

Déclaration	Signification	Cas d'usage
virtual void f()	Peut être surchargée	Comportement par défaut
virtual void f() = 0	DOIT être surchargée	Interface/abstraction
void f() override	Surcharge (vérifiée par compilateur)	Classes dérivées
void f() final	Interdit la surcharge ultérieure	Optimisation / design
virtual ~Base() = default	Destructeur polymorphique	Toute base

4.3 Interface en C++

```
// Interface = classe abstraite pure (0 données, 0 corps)
class ILogger {
public:
    virtual void log(const std::string& msg) = 0;
    virtual void flush() = 0;
    virtual ~ILogger() = default;
};

// Implémenter plusieurs interfaces (héritage multiple)
class FileLogger : public ILogger, public IFormattable {
public:
    void log(const std::string& msg) override { /* ... */ }
    void flush() override { /* ... */ }
    std::string format() const override { /* ... */ }
};
```

4.4 RTTI — identification du type runtime

```
// dynamic_cast : cast sécurisé polymorphique
Animal* a = new Chien("Rex");

// Pour les pointeurs (retourne nullptr si échec)
if (Chien* d = dynamic_cast<Chien*>(a)) {
    d->aboyer(); // utilisation sûre
}

// Pour les références (throw std::bad_cast si échec)
try {
    Chien& d = dynamic_cast<Chien&>(*a);
} catch (const std::bad_cast&) { }

// typeid : obtenir le type exact
std::cout << typeid(*a).name(); // "Chien"
```

5. Mémoire et Smart Pointers

5.1 Comparatif des smart pointers

	unique_ptr<T>	shared_ptr<T>	weak_ptr<T>
Propriété	Exclusive	Partagée	Aucune
Copiable	✗	✓	✓
Déplaçable	✓	✓	✓
Overhead	~0	Compteur atomique	Compteur

	<code>unique_ptr<T></code>	<code>shared_ptr<T></code>	<code>weak_ptr<T></code>
Cas d'usage	Par défaut	Partage explicite	Observer, cycles
Création	<code>make_unique<T>()</code>	<code>make_shared<T>()</code>	Depuis <code>shared_ptr</code>

5.2 RAI — le principe fondamental

Règle : chaque ressource est liée à la durée de vie d'un objet C++

Acquisition → constructeur

Libération → destructeur (toujours appelé, même si exception)

Ressources concernées :

- ✓ Mémoire heap (`unique_ptr`, `vector`, `string`)
- ✓ Fichiers (`ifstream`, `ofstream`)
- ✓ Mutex (`lock_guard`, `unique_lock`, `scoped_lock`)
- ✓ Connexions réseau/base de données
- ✓ Handles systèmes (`HANDLE`, `FILE*`, descripteurs)

5.3 Cas d'usage des smart pointers

```
// unique_ptr : propriété exclusive
auto ptr = std::make_unique<Voiture>("Tesla");
auto autre = std::move(ptr); // transfert de propriété
// ptr est maintenant nullptr

// shared_ptr : propriété partagée
auto sp1 = std::make_shared<Connexion>("db://...");
auto sp2 = sp1; // partage, compteur = 2
// Libéré quand compteur = 0

// weak_ptr : observer sans posséder
std::weak_ptr<Session> sessionRef = sessionPtr;
if (auto session = sessionRef.lock()) { // accès sécurisé
    session->utiliser();
}
// Évite les cycles : Parent ↔ Enfant
```

5.4 Move Semantics

```
// std::move : cast vers rvalue reference (transfert)
std::string a = "Hello";
std::string b = std::move(a); // b = "Hello", a = "" (valide mais vide)

// Move constructor : O(1) même pour les gros objets
std::vector<int> v(1000000);
std::vector<int> w = std::move(v); // O(1) – juste swap des pointeurs internes

// Règle : après std::move, n'utilisez plus l'objet source
// (sauf pour le réaffecter ou le détruire)
```

6. STL — Conteneurs

6.1 Tableau comparatif global

Conteneur	Accès	Insert début	Insert fin	Insert milieu	Find	Trié	Unique
vector<T>	O(1)	O(n)	O(1)*	O(n)	O(n)	Non	Non
deque<T>	O(1)	O(1)	O(1)	O(n)	O(n)	Non	Non
list<T>	O(n)	O(1)	O(1)	O(1)†	O(n)	Non	Non
array<T,N>	O(1)	—	—	O(n)	O(n)	Non	Non
map<K,V>	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	Oui	Oui
unordered_map<K,V>	O(1)*	O(1)*	O(1)*	O(1)*	O(1)*	Non	Oui
set<T>	O(log n)	O(log n)	O(log n)	O(log n)	O(log n)	Oui	Oui
unordered_set<T>	O(1)*	O(1)*	O(1)*	O(1)*	O(1)*	Non	Oui
priority_queue<T>	O(1) top	O(log n)	O(log n)	—	O(n)	Partiel	Non

*Amorti | †Nécessite un itérateur

6.2 Guide de choix du conteneur

```
Je veux stocker des éléments...
├─ En accédant par index → vector<T> (sauf insertions fréquentes au milieu)
├─ En itérant souvent → vector<T> (meilleure localité cache)
├─ Avec insertion fréquente aux deux extrémités → deque<T>
├─ Avec insertion/suppression fréquente n'importe où → list<T> (rare en pratique)
|
├─ Avec accès par clé, trié → map<K,V>
├─ Avec accès par clé, rapide (O(1)) → unordered_map<K,V>
|
├─ Unique + trié → set<T>
├─ Unique + rapide → unordered_set<T>
|
├─ LIFO (dernier entré, premier sorti) → stack<T> (sur deque)
├─ FIFO (premier entré, premier sorti) → queue<T> (sur deque)
└─ Par priorité → priority_queue<T>
```

6.3 Opérations clés par conteneur

```
// — vector —
std::vector<int> v;
v.push_back(42);           // ajouter en fin
v.emplace_back(42);        // construire en place (plus efficace)
v.insert(v.begin(), 0);    // insérer en début (O(n))
v.erase(v.begin() + 2);    // supprimer à l'index 2
v.reserve(1000);           // pré-allouer (évite les réallocations)
v.resize(10, 0);           // redimensionner (remplit avec 0)
v.clear();                 // vider (capacity inchangée)
v.shrink_to_fit();         // libérer la capacité excédentaire
v.at(5);                   // accès sécurisé (throw si hors bornes)

// — map —
std::map<std::string, int> m;
m["clé"] = 42;             // insertion ou mise à jour
m.insert({"clé", 42});     // insertion seulement (pas d'écrasement)
m.emplace("clé", 42);      // construction en place
m.erase("clé");            // suppression
m.count("clé");            // 0 ou 1
m.contains("clé");         // C++20 : booléen
auto it = m.find("clé");   // itérateur (end() si absent)
auto [it2, ok] = m.try_emplace("clé", 42); // insère seulement si absent

// — unordered_map —
std::unordered_map<std::string, int> um;
um.reserve(10000);         // éviter le rehashing
um.max_load_factor(0.5);   // moins de collisions
```

7. STL — Algorithmes

7.1 Référence des algorithmes les plus utilisés

Algorithme	Complexité	Description	Exemple
sort	O(n log n)	Tri introsort (non-stable)	std::sort(v.begin(), v.end())
stable_sort	O(n log n)	Tri stable	std::stable_sort(v.begin(), v.end())
find	O(n)	Première occurrence	std::find(v.begin(), v.end(), 42)
find_if	O(n)	Première correspondance	std::find_if(v.begin(), v.end(), pred)
count	O(n)	Nombre d'occurrences	std::count(v.begin(), v.end(), 42)

Algorithme	Complexité	Description	Exemple
count_if	O(n)	Nombre correspondants	std::count_if(v.begin(), v.end(), pred)
binary_search	O(log n)	Existence (tableau trié)	std::binary_search(v.begin(), v.end(), 42)
lower_bound	O(log n)	Borne inférieure	std::lower_bound(v.begin(), v.end(), 42)
upper_bound	O(log n)	Borne supérieure	std::upper_bound(v.begin(), v.end(), 42)
transform	O(n)	Appliquer une fonction	std::transform(v.begin(), v.end(), out, fn)
accumulate	O(n)	Réduction	std::accumulate(v.begin(), v.end(), 0)
reduce	O(n)	Réduction parallélisable (C++17)	std::reduce(v.begin(), v.end(), 0)
copy	O(n)	Copier	std::copy(v.begin(), v.end(), out)
copy_if	O(n)	Copier si prédicat	std::copy_if(v.begin(), v.end(), out, pred)
fill	O(n)	Remplir	std::fill(v.begin(), v.end(), 0)
generate	O(n)	Générer	std::generate(v.begin(), v.end(), fn)
remove	O(n)	Retirer (idiome erase-remove)	v.erase(std::remove(v.begin(), v.end(), 42), v.end())
remove_if	O(n)	Retirer si prédicat	v.erase(std::remove_if(..., pred), v.end())
unique	O(n)	Supprimer doublons consécutifs	v.erase(std::unique(v.begin(), v.end()), v.end())
reverse	O(n)	Inverser	std::reverse(v.begin(), v.end())
rotate	O(n)	Rotation	std::rotate(v.begin(), v.begin()+k, v.end())
max_element	O(n)	Itérateur vers le max	*std::max_element(v.begin(), v.end())
min_element	O(n)	Itérateur vers le min	*std::min_element(v.begin(), v.end())
minmax_element	O(n)	Paire min+max	auto [lo, hi] = std::minmax_element(v.begin(), v.end())
partition	O(n)	Partitionner	std::partition(v.begin(), v.end(), pred)
nth_element	O(n)	Pivot à la bonne position	std::nth_element(v.begin(), v.begin()+k, v.end())

Algorithme	Complexité	Description	Exemple
for_each	O(n)	Appliquer un effet de bord	std::for_each(v.begin(), v.end(), fn)
any_of	O(n)	Au moins un vérifie	std::any_of(v.begin(), v.end(), pred)
all_of	O(n)	Tous vérifient	std::all_of(v.begin(), v.end(), pred)
none_of	O(n)	Aucun ne vérifie	std::none_of(v.begin(), v.end(), pred)

7.2 Idiomes courants

```
// — Erase-Remove —
v.erase(std::remove(v.begin(), v.end(), val), v.end());
v.erase(std::remove_if(v.begin(), v.end(), pred), v.end());
std::erase(v, val);           // C++20 : plus simple
std::erase_if(v, pred);       // C++20

// — Transformer en place —
std::transform(v.begin(), v.end(), v.begin(), [](int x){ return x*2; });

// — Accumuler en string —
std::string result = std::accumulate(
    str.begin(), str.end(), std::string{},
    [](std::string acc, const std::string& s) {
        return acc + (acc.empty() ? "" : ", ") + s;
    });

// — Partition + copie —
std::vector<int> pairs, impairs;
std::partition_copy(v.begin(), v.end(),
    std::back_inserter(pairs),
    std::back_inserter(impairs),
    [](int x){ return x % 2 == 0; });
```

8. Templates

8.1 Syntaxes de base

```
// — Template de fonction —
template<typename T>
T max(T a, T b) { return a > b ? a : b; }

// Plusieurs paramètres de type
template<typename T, typename U>
auto additionner(T a, U b) -> decltype(a + b) { return a + b; }

// Paramètre non-type
template<int N>
std::array<int, N> creerTableau() { return {}; }

// — Template de classe —
template<typename T, size_t Capacite = 100>
class Pile {
    T elements[Capacite];
    size_t sommet{0};
public:
    void push(const T& v) { elements[sommet++] = v; }
    T pop() { return elements[--sommet]; }
};

// — Spécialisation totale —
template<> class Pile<bool, 64> { /* version optimisée */ };

// — Alias de template —
template<typename T>
using Vec = std::vector<T>;
Vec<int> v; // = std::vector<int>
```

8.2 Concepts C++20

```
#include <concepts>

// Concepts prédéfinis utiles
std::same_as<T, U>           // même type
std::derived_from<T, Base>   // dérivé de Base
std::integral<T>            // type entier
std::floating_point<T>       // flottant
std::arithmetic<T>          // integral || floating_point
std::convertible_to<From, To>
std::invocable<F, Args...>
std::ranges::range<R>

// Définir un concept
template<typename T>
concept Comparable = requires(T a, T b) {
    { a < b } -> std::convertible_to<bool>;
    { a == b } -> std::convertible_to<bool>;
};

// Utiliser un concept (3 syntaxes)
template<Comparable T> T max(T a, T b);
template<typename T> requires Comparable<T> T max(T a, T b);
auto max(Comparable auto a, Comparable auto b);
```

9. Exceptions

9.1 Hiérarchie standard

```
std::exception
├─ std::logic_error
│   ├─ std::invalid_argument
│   ├─ std::out_of_range
│   ├─ std::length_error
│   └─ std::domain_error
├─ std::runtime_error
│   ├─ std::overflow_error
│   ├─ std::underflow_error
│   └─ std::range_error
├─ std::bad_alloc          ← new a échoué
├─ std::bad_cast           ← dynamic_cast sur référence a échoué
└─ std::bad_typeid         ← typeid(nullptr)
```

9.2 Garanties d'exception

Garantie	Description	Obtenir comment
Nothrow	Ne lance jamais d'exception	noexcept , opérations triviales

Garantie	Description	Obtenir comment
Forte	Succès ou état inchangé (rollback)	Copy-and-swap, opérations atomiques
Basique	Pas de fuite, état valide mais non-spécifié	Libérer les ressources, pas de corruption
Aucune	Tout peut arriver	Code non-sécurisé (ne pas produire)

9.3 Bonnes pratiques

```
// ✅ Attraper par référence constante
try { risque(); }
catch (const std::invalid_argument& e) { /* spécifique */ }
catch (const std::exception& e) { /* général */ }
catch (...) { /* catch-all (rare) */ }

// ✅ Relancer sans copie
catch (const std::exception& e) {
    log(e.what());
    throw; // relance l'exception originale (pas une copie)
}

// ✅ Exceptions personnalisées
class MonErreur : public std::runtime_error {
public:
    explicit MonErreur(const std::string& msg)
        : std::runtime_error(msg) {}
};

// ❌ Ne jamais lancer depuis un destructeur
~MaClasse() noexcept {
    try { nettoyer(); }
    catch (...) { /* avaler silencieusement */ }
}
```

10. C++ Moderne

10.1 auto et déduction de type

```
auto i = 42;           // int
auto d = 3.14;         // double
auto s = std::string{"hi"}; // string (pas const char*)
auto& r = x;           // int& (référence)
const auto& cr = x;    // const int&
auto* p = &x;          // int*

// decltype : type exact d'une expression
decltype(x) y = x;     // même type que x
decltype(auto) f() { return x; } // préserve références

// Structured bindings (C++17)
auto [min, max] = std::minmax(a, b);
auto& [key, val] = *map.begin();
for (auto& [k, v] : maMap) { }
```

10.2 Lambdas — syntaxe complète

```
// [capture](paramètres) mutable? noexcept? -> type_retour { corps }

auto f1 = []() { }; // aucune capture
auto f2 = [x]() { return x; }; // capture x par valeur
auto f3 = [&x]() { x++; }; // capture x par référence
auto f4 = [=]() { return x + y; }; // tout par valeur
auto f5 = [&]() { x++; y++; }; // tout par référence
auto f6 = [x, &y]() { return x + y; }; // mixte
auto f7 = [v = std::move(vec)]() { }; // init capture (move)
auto f8 = [](auto x) { return x * 2; }; // générique (C++14)
auto f9 = []() mutable { static int n=0; return n++; };
```

10.3 std::optional, variant, any

```
// — optional —
std::optional<int> chercher(const std::string& s) {
    if (existe) return valeur;
    return std::nullopt;
}

auto r = chercher("key");
if (r) { use(*r); }          // déréférencement
r.value_or(-1);              // valeur par défaut
r.has_value();               // vérification

// — variant —
std::variant<int, double, std::string> v;
v = 42;
v = "hello";
std::get<std::string>(v);     // accès typé (throw si mauvais type)
std::get_if<int>(&v);         // pointeur (nullptr si mauvais type)
std::visit([](auto&& val) { }, v); // visitor

// — any —
std::any a = 42;
a = std::string("hello");
std::any_cast<std::string>(a); // throw si mauvais type
std::any_cast<std::string*>(&a); // pointeur (nullptr si mauvais type)
```

10.4 Ranges C++20 — vues chaînables

```
#include <ranges>

auto resultat = v
    | std::views::filter([](int x) { return x > 0; }) // filtrer
    | std::views::transform([](int x) { return x * 2; }) // transformer
    | std::views::take(10) // limiter
    | std::views::reverse; // inverser

// Vues courantes
std::views::iota(0, 100) // séquence 0..99
std::views::filter(pred) // filtrer
std::views::transform(fn) // transformer
std::views::take(n) // prendre n éléments
std::views::drop(n) // ignorer n éléments
std::views::reverse // inverser
std::views::enumerate // (index, valeur)
std::views::zip(r1, r2) // combiner deux ranges
std::views::join // aplatir range de ranges
std::views::split(delim) // découper
```

11. Concurrency

11.1 Comparatif des primitives de synchronisation

Primitive	Usage	Coût	Bloquant
std::mutex	Exclusion mutuelle	Moyen	Oui
std::shared_mutex	Lecture multiple / écriture exclusive	Moyen	Oui
std::atomic<T>	Variables simples sans mutex	Faible	Non
std::condition_variable	Attente d'une condition	Moyen	Oui
std::counting_semaphore	Limiter la concurrence	Moyen	Oui
std::latch	Barrière usage unique	Faible	Oui
std::barrier	Barrière cyclique	Moyen	Oui

11.2 Locks — tableau de choix

Lock	Usage	Déverrouillage manuel
lock_guard<mutex>	Lock simple, scope-based	Non
unique_lock<mutex>	Lock flexible (timed, deferred)	Oui
shared_lock<shared_mutex>	Lock de lecture	Oui
scoped_lock<m1, m2>	Verrouiller plusieurs mutex (sans deadlock)	Non

11.3 Memory Order — référence

Order	Utilisation
memory_order_relaxed	Compteurs sans synchronisation (perf max)
memory_order_acquire	Après ce load, toutes les écritures précédant un release sont visibles
memory_order_release	Avant ce store, toutes les écritures sont visibles pour un acquire suivant
memory_order_acq_rel	acquire + release (read-modify-write)
memory_order_seq_cst	Ordre total global (défaut, le plus sûr)

11.4 Patterns concurrents courants

```
// — Producteur / Consommateur —
std::queue<T> q;
std::mutex mtx;
std::condition_variable cv;

void produire(T val) {
    { std::lock_guard lock(mtx); q.push(val); }
    cv.notify_one();
}

T consommer() {
    std::unique_lock lock(mtx);
    cv.wait(lock, [&]{ return !q.empty(); });
    T val = std::move(q.front()); q.pop();
    return val;
}

// — Double-checked locking (Singleton thread-safe) —
static std::once_flag flag;
static std::unique_ptr<T> instance;
static T& get() {
    std::call_once(flag, []{ instance = std::make_unique<T>(); });
    return *instance;
}

// — Read-Write Lock —
std::shared_mutex rwmtx;
void lire() { std::shared_lock lock(rwmtx); /* ... */ }
void ecrire() { std::unique_lock lock(rwmtx); /* ... */ }
```

12. Design Patterns

12.1 Tableau de référence

Pattern	Catégorie	Problème résolu	Implémentation clé
Singleton	Créationnel	Instance unique et globale	call_once + unique_ptr statique
Factory Method	Créationnel	Créer sans connaître le type	Fonction retournant unique_ptr<Base>
Abstract Factory	Créationnel	Familles de produits	Interface avec plusieurs factory methods
Builder	Créationnel	Construction complexe et configurable	Méthodes chaînables, retournent *this
Prototype	Créationnel	Cloner des objets	Méthode virtuelle clone()

Pattern	Catégorie	Problème résolu	Implémentation clé
Adapter	Structurel	Deux interfaces incompatibles	Wrapper implémentant la cible
Decorator	Structurel	Ajouter des responsabilités	Même interface, délègue + enrichit
Composite	Structurel	Hiérarchie partie-tout	Même interface pour feuilles et nœuds
Facade	Structurel	Simplifier un sous-système	Classe de coordination unique
Proxy	Structurel	Contrôle d'accès	Même interface, interception
Flyweight	Structurel	Partager les données immuables	Pool d'objets partagés
Observer	Comportemental	Notification $N \rightarrow M$	<code>vector<function<void(Event)>></code>
Strategy	Comportemental	Algorithme interchangeable	<code>function<></code> ou interface virtuelle
Command	Comportemental	Action encapsulée + undo	Objet avec <code>execute()</code> et <code>undo()</code>
State	Comportemental	Comportement selon l'état	État = objet, délégation
Template Method	Comportemental	Algorithme avec trous	Méthode publique non-virtuelle + virtuelles protégées
Chain of Responsibility	Comportemental	Chaîne de traitements	Chaque handler passe au suivant
Iterator	Comportemental	Parcourir sans connaître la structure	<code>begin()/end()</code> + opérateurs
Mediator	Comportemental	Découpler des composants	Objet central de coordination

13. CMake

13.1 CMakeLists.txt type

```
cmake_minimum_required(VERSION 3.20)
project(MonProjet VERSION 1.0.0 LANGUAGES CXX)

# Standard
set(CMAKE_CXX_STANDARD 17)
set(CMAKE_CXX_STANDARD_REQUIRED ON)
set(CMAKE_CXX_EXTENSIONS OFF)

# Options
option(BUILD_TESTS "Compiler les tests" ON)
option(ENABLE_SANITIZERS "Activer ASan/UBSan" OFF)

# Warnings
add_compile_options(-Wall -Wextra -Wpedantic)
if(ENABLE_SANITIZERS)
    add_compile_options(-fsanitize=address,undefined)
    add_link_options(-fsanitize=address,undefined)
endif()

# Bibliothèque
add_library(monlib STATIC src/calcul.cpp src/utils.cpp)
target_include_directories(monlib PUBLIC include)
target_compile_features(monlib PUBLIC cxx_std_17)

# Exécutable
add_executable(monapp src/main.cpp)
target_link_libraries(monapp PRIVATE monlib)

# Dépendances externes
include(FetchContent)
FetchContent_Declare(Catch2
    GIT_REPOSITORY https://github.com/catchorg/Catch2.git
    GIT_TAG v3.4.0)
FetchContent_MakeAvailable(Catch2)

# Tests
if(BUILD_TESTS)
    enable_testing()
    add_subdirectory(tests)
endif()
```

13.2 Commandes CMake courantes

```
# Configuration
cmake -B build                                # configurer dans build/
cmake -B build -DCMAKE_BUILD_TYPE=Debug      # mode debug
cmake --preset debug                          # avec CMakePresets.json

# Compilation
cmake --build build                          # compiler
cmake --build build --parallel               # en parallèle
cmake --build build --target monapp          # une cible précise

# Tests
cd build && ctest                             # tous les tests
ctest --output-on-failure                   # afficher si échec
ctest --test-dir build -R "^test_calcul"    # filtrer par regex

# Installation/Packaging
cmake --install build --prefix /usr/local
cd build && cpack -G DEB                       # package Debian
```

14. Tests

14.1 Catch2 — référence rapide

```
#include <catch2/catch_test_macros.hpp>

TEST_CASE("Nom du test", "[tag1][tag2]") {
    // REQUIRE : arrête le test si faux
    REQUIRE(2 + 2 == 4);
    REQUIRE_FALSE(false);
    REQUIRE_THROWS_AS(f(), std::invalid_argument);
    REQUIRE_NOTHROW(g());
    REQUIRE_THAT(str, Catch::Matchers::Contains("hello"));

    // CHECK : continue même si faux (note l'échec)
    CHECK(valeur == attendu);

    SECTION("sous-test A") { /* ... */ }
    SECTION("sous-test B") { /* ... */ }
}

// Tests paramétrés
TEST_CASE("Fibonacci") {
    auto [n, expected] = GENERATE(table<int, int>({
        {0, 0}, {1, 1}, {10, 55}
    }));
    REQUIRE(fibonacci(n) == expected);
}

// Benchmark
TEST_CASE("Benchmark tri") {
    BENCHMARK("std::sort") {
        auto v = données;
        std::sort(v.begin(), v.end());
        return v;
    };
}
```

14.2 GoogleTest — référence rapide

```
#include <gtest/gtest.h>

// Test simple
TEST(NomSuite, NomTest) {
    EXPECT_EQ(2 + 2, 4);      // continue si faux
    ASSERT_EQ(2 + 2, 4);      // arrête si faux
    EXPECT_NEAR(a, b, 0.001); // flottants
    EXPECT_THROW(f(), std::exception);
    EXPECT_NO_THROW(g());
}

// Fixture (setup/teardown)
class MaFixture : public ::testing::Test {
protected:
    Calculatrice calc;
    void SetUp() override { calc.reset(); }
    void TearDown() override { }
};

TEST_F(MaFixture, Addition) { EXPECT_EQ(calc.add(2,3), 5); }

// Tests paramétrés
class TestPremier : public ::testing::TestWithParam<std::pair<int, bool>> {};
TEST_P(TestPremier, VerifPremier) {
    auto [n, expected] = GetParam();
    EXPECT_EQ(estPremier(n), expected);
}
INSTANTIATE_TEST_SUITE_P(Valeurs, TestPremier,
    ::testing::Values(std::pair{2, true}, std::pair{4, false}));
```

14.3 Couverture de code

```
# Compilation avec gcov
g++ -fprofile-arcs -ftest-coverage -g -o tests tests/*.cpp

# Rapport texte
gcovr --root . --print-summary

# Rapport HTML
gcovr --root . --html-details coverage.html
firefox coverage.html

# Exiger minimum 80%
gcovr --fail-under-line 80 .
```

15. Sécurité

15.1 Vulnérabilités courantes et corrections

Vulnérabilité	CWE	Exemple dangereux	Correction
Buffer Overflow	CWE-120	strcpy(buf, input)	std::string OU strncpy
Stack BOF	CWE-121	char buf[32]; gets(buf)	std::getline
Use-After-Free	CWE-416	delete p; *p = 1	Smart pointers
Double Free	CWE-415	delete p; delete p	Smart pointers
Integer Overflow	CWE-190	INT_MAX + 1	Vérifier avant opération
SQL Injection	CWE-89	"SELECT * WHERE id=" + id	Requêtes préparées
Path Traversal	CWE-22	/www/html/ + chemin_user	Valider et normaliser
Format String	CWE-134	printf(user_input)	printf("%s", user_input)
NULL Deref	CWE-476	p->f() sans vérif	Vérifier avant déréférencement

15.2 Outils de détection

```
# Compilation sécurisée
g++ -O2 -D_FORTIFY_SOURCE=2 -fstack-protector-strong \
    -fPIE -pie -Wl,-z,relro,-z,now main.cpp

# Runtime (développement)
g++ -fsanitize=address,undefined -g -o prog main.cpp # ASan + UBSan
g++ -fsanitize=thread -g -o prog main.cpp           # TSan (data race)

# Analyse statique
cppcheck --enable=all src/
clang-tidy src/*.cpp -- -std=c++17 -I include/
scan-build cmake --build build          # Clang Static Analyzer

# Fuzzing
clang++ -fsanitize=fuzzer,address -g -o fuzzer fuzz.cpp
./fuzzer corpus/ -max_len=1000 -runs=1000000
```

16. Performance

16.1 Règles d'optimisation

- Règle d'or : MESURER AVANT D'OPTIMISER
- Ordre d'impact (du plus au moins important) :
- 1. Choisir le bon algorithme ($O(n)$ vs $O(n^2)$)
 - 2. Choisir la bonne structure de données
 - 3. Éviter les allocations inutiles
 - 4. Améliorer la localité cache
 - 5. Utiliser le parallélisme
 - 6. Micro-optimisations (SIMD, etc.)

16.2 Techniques d'optimisation

Technique	Speedup typique	Description
reserve() sur vector	2-5×	Évite les réallocations
emplace_back vs push_back	10-30%	Construction en place
string_view VS string&	20-50%	Évite les copies
Move semantics	10-100×	Déplacer au lieu de copier
Cache-friendly layout (SoA)	2-10×	Données contiguës
LTO (-flto)	5-20%	Optimisations cross-module
PGO	10-30%	Optimisations guidées par profil
std::execution::par	2-8×	Algorithmes parallèles
ThreadPool (4 cœurs)	3-4×	Tâches indépendantes

16.3 Profiling rapide

```
# Compiler avec symboles
g++ -g -O2 -fno-omit-frame-pointer -o app main.cpp

# perf (Linux)
perf stat ./app           # compteurs CPU
perf record -g ./app      # enregistrer l'exécution
perf report               # analyser

# Valgrind Callgrind
valgrind --tool=callgrind ./app
callgrind_annotate callgrind.out.*

# Google Benchmark (dans le code)
BENCHMARK(BM_maFonction)->RangeMultiplier(10)->Range(100, 100000);
BENCHMARK_MAIN();
```

17. Bonnes pratiques

17.1 Les 20 règles d'or

#	Règle	Justification
1	Zéro <code>new</code> / <code>delete</code> <code>raw</code>	<code>unique_ptr</code> , <code>make_unique</code> évitent les fuites
2	RAII pour toute ressource	Libération garantie même en cas d'exception
3	<code>const</code> partout où possible	Documente l'intention, permet les optimisations
4	<code>[[nodiscard]]</code> sur les fonctions critiques	Oblige à gérer le résultat
5	<code>override</code> sur toutes les surcharges	Détecte les erreurs de signature
6	Destructeur <code>virtual</code> sur les bases	Évite les destructions incomplètes
7	<code>noexcept</code> quand garanti	Permet des optimisations STL
8	Explicit sur les constructeurs mono-argument	Évite les conversions implicites surprenantes
9	Pas de <code>using namespace std</code> dans les headers	Pollue l'espace de noms des utilisateurs
10	<code>#pragma once</code> dans chaque header	Protection simple contre les inclusions multiples
11	Paramètres <code>const T&</code> pour les objets	Évite les copies inutiles
12	Range-based for plutôt qu'index	Plus lisible, moins d'erreurs de borne
13	<code>auto</code> pour les itérateurs et types complexes	Réduit la verbosité sans perdre la clarté
14	<code>std::optional</code> plutôt que <code>-1</code> ou <code>nullptr</code>	Exprime explicitement l'absence de valeur
15	<code>lock_guard</code> plutôt que <code>lock/unlock</code> manual	RAII pour les mutex
16	<code>static_assert</code> pour les invariants de compilation	Erreurs plus tôt et plus claires
17	Tests en parallèle du développement	Détection précoce des bugs
18	Commenter le POURQUOI, pas le QUOI	Le code dit déjà ce qu'il fait
19	Mesurer avant d'optimiser	Les intuitions sur les perfs sont souvent fausses
20	Un seul rôle par classe (SRP)	Plus facile à tester, modifier, réutiliser

17.2 Anti-patterns à éviter

```
// ✗ God class (tout en un)
class Application { /* 500 méthodes */ };

// ✗ Magic numbers
if (score >= 42) { } // → constexpr int SCORE_VALIDATION = 42;

// ✗ Primitive obsession
void creer(string, string, string, string, int, bool, double);
// → struct UtilisateurParams { ... };

// ✗ Deep nesting (flèche de code)
if (a) { if (b) { if (c) { /* ... */ } } }
// → Guard clauses (return early)

// ✗ Commentaire inutile
i++; // incrémenter i

// ✗ Variable globale mutable
int g_compteur = 0; // → passer en paramètre ou encapsuler

// ✗ Exception pour le flux normal
for(;;) { try { get(i++); } catch(...) { break; } }

// ✗ Raw loop quand un algorithme STL existe
// → std::transform, std::accumulate, std::find_if
```

18. Outils

18.1 Compilateurs et flags essentiels

```
# Développement : debug + warnings + sanitizers
g++ -std=c++17 -g -O0 -Wall -Wextra -Wpedantic \
    -fsanitize=address,undefined \
    -fno-omit-frame-pointer \
    main.cpp -o prog

# Production : optimisation + sécurité
g++ -std=c++17 -O2 -DNDEBUG \
    -D_FORTIFY_SOURCE=2 \
    -fstack-protector-strong \
    -fPIE -pie \
    -Wl,-z,relro,-z,now \
    main.cpp -o prog

# Profiling
g++ -std=c++17 -g -O2 -fno-omit-frame-pointer main.cpp -o prog
```

18.2 Tableau des outils

Outil	Rôle	Commande clé
g++ / clang++	Compilateur	<code>g++ -std=c++17 -Wall</code>
cmake	Build system	<code>cmake -B build && cmake --build build</code>
make / ninja	Backend de build	<code>cmake -G Ninja</code>
ctest	Test runner	<code>ctest --output-on-failure</code>
valgrind	Fuites mémoire	<code>valgrind --leak-check=full ./prog</code>
AddressSanitizer	Erreurs mémoire	<code>-fsanitize=address</code>
ThreadSanitizer	Data races	<code>-fsanitize=thread</code>
UBSanitizer	UB runtime	<code>-fsanitize=undefined</code>
cppcheck	Analyse statique	<code>cppcheck --enable=all src/</code>
clang-tidy	Linting + modernisation	<code>clang-tidy src/*.cpp --</code>
clang-format	Formatage	<code>clang-format -i src/*.cpp</code>
gcovr	Couverture	<code>gcovr --html coverage.html</code>
perf	Profiling CPU	<code>perf record -g ./prog</code>
callgrind	Profiling fin	<code>valgrind --tool=callgrind ./prog</code>
doxygen	Documentation	<code>doxygen Doxyfile</code>
conan / vcpkg	Gestion paquets	<code>conan install . --build=missing</code>

18.3 Configuration clang-format recommandée

```
# .clang-format
BasedOnStyle: Google
IndentWidth: 4
ColumnLimit: 100
AllowShortFunctionsOnASingleLine: Inline
AllowShortIfStatementsOnASingleLine: false
SortIncludes: CaseSensitive
IncludeBlocks: Regroup
```

18.4 Configuration clang-tidy recommandée

```
# .clang-tidy
Checks: >
  bugprone-*,
  cppcoreguidelines-*,
  modernize-*,
  performance-*,
  readability-*,
  -modernize-use-trailing-return-type,
  -cppcoreguidelines-avoid-magic-numbers

WarningsAsErrors: "bugprone-*,performance-*
```

Index rapide — Trouver ce dont vous avez besoin

Je cherche...	Section	Code exemple
Créer un tableau dynamique	§6	<code>std::vector<int> v; v.push_back(42);</code>
Créer une map clé→valeur	§6	<code>std::map<string,int> m; m["x"] = 1;</code>
Trouver un élément	§7	<code>auto it = std::find(v.begin(), v.end(), val)</code>
Trier un vecteur	§7	<code>std::sort(v.begin(), v.end())</code>
Smart pointer exclusif	§5	<code>auto p = std::make_unique<T>(args)</code>
Smart pointer partagé	§5	<code>auto p = std::make_shared<T>(args)</code>
Valeur ou absence	§10	<code>std::optional<int> opt = trouver("x")</code>
Lambda simple	§10	<code>[x](int y) { return x + y; }</code>
Créer un thread	§11	<code>std::thread t(fn, args); t.join();</code>
Protéger une section	§11	<code>std::lock_guard<std::mutex> lock(mtx);</code>
Résultat asynchrone	§11	<code>auto f = std::async(fn); int r = f.get();</code>
Template de fonction	§8	<code>template<typename T> T max(T a, T b)</code>
Lancer une exception	§9	<code>throw std::invalid_argument("msg")</code>
Configurer CMake	§13	Voir §13.1
Écrire un test Catch2	§14	<code>TEST_CASE("nom") { REQUIRE(2+2 == 4); }</code>
Détecter une fuite	§15	<code>valgrind --leak-check=full ./prog</code>
Mesurer les perfs	§16	<code>perf record -g ./prog && perf report</code>

Mémento C++ — Formation 5 jours · Mis à jour pour C++17/20

cppreference.com pour la référence exhaustive · isocpp.org pour les guidelines